# COMPOUND & NESTED CONDITIONS

Block 15 • Turning policy language into readable decision logic

```
if (
    status == "active"
    and error_count == 0
    and is_approved
):
    print("Ready")
else:
    print("Review")
```


**Today: combine conditions without turning your program into spaghetti.**

# Where this block fits

### Already learned

- Values and variables
- Comparisons produce True / False
- if and else choose a path
- elif selects one outcome

### Today adds

- and / or / not inside conditions
- Nested decisions
- Decision tables
- Policy rules translated to code

### Goal

**Write decision logic that is correct, explainable, and readable enough to maintain later.**

**Readable logic is not a luxury. It is how you avoid bugs.**

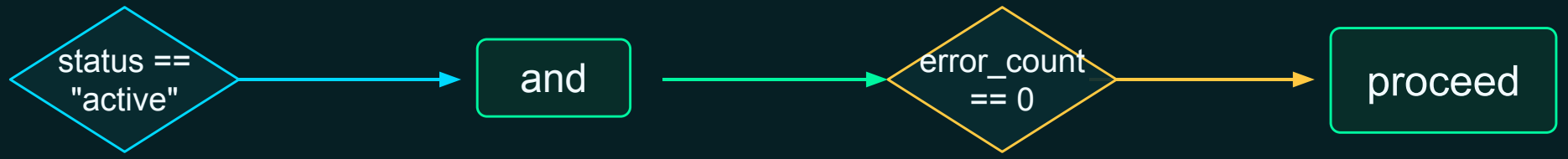
CONCEPT

# Compound conditions combine multiple requirements

```
if status == "active" and error_count == 0:
    print("Record may proceed")
```

**Plain language**

The record may proceed only when it is active and has no errors.




**and means every requirement must pass.**

TOOLS

# and, or, and not

**and**

**Both sides must be True.**

```
age >= 18 and has_badge
```

stricter

**or**

**At least one side must be True.**

```
role == "admin" or role == "owner"
```

flexible

**not**

**Flips True to False, or False to True.**

```
not system_locked
```

opposite

**Use these to make code sound like the rule you are implementing.**

# Parentheses turn long logic into a checklist

```
if status == "active" and completion_percentage >= 95 and error_count == 0 and is_approved:
    print("Ready for publication")
```

**Technically valid, but hard to scan.**

```
if (
    status == "active"
    and completion_percentage >= 95
    and error_count == 0
    and is_approved
):
    print("Ready for publication")
```

Why this helps

- One requirement per line
- Easy to add or remove a check
- Fewer mistakes when reading aloud
- No backslash needed

**For long conditions, prefer boring and readable.**

PATTERN

# Multiple acceptable values

```
if file_extension == "csv" or
file_extension == "json":
    print("Supported format")
```

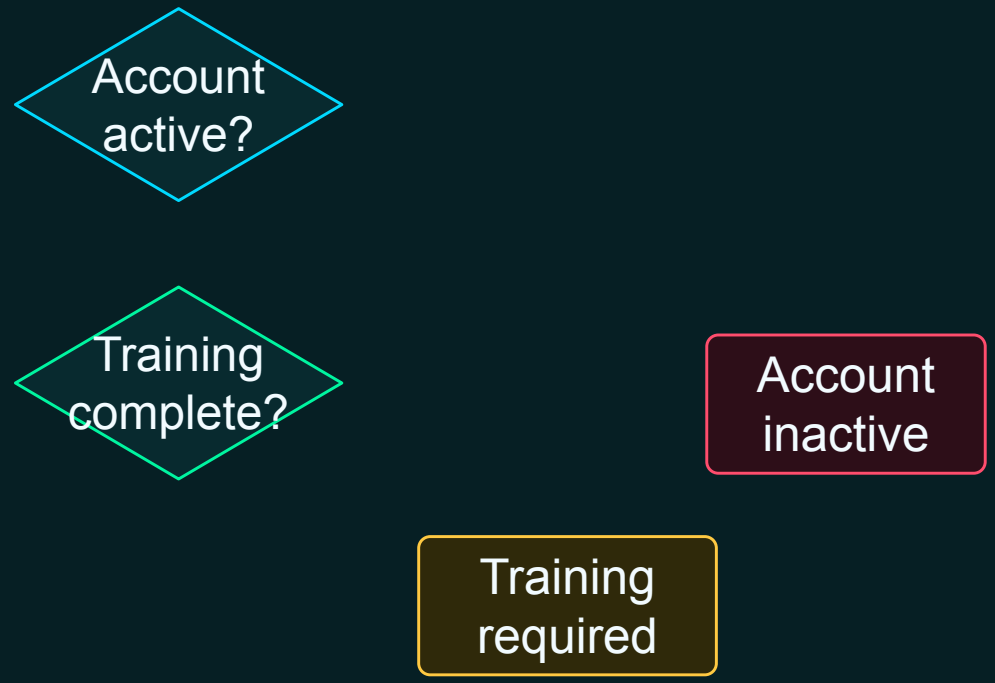
**This condition means: CSV is okay, JSON is okay, anything else is not.**

| file_extension | csv? | json? | accepted? |
|---|---|---|---|
| csv | True | False | True |
| json | False | True | True |
| xlsx | False | False | False |

**or is useful when several options are acceptable.**

# A nested if handles dependent decisions

```
if account_active:
    if training_complete:
        print("Access granted")
    else:
        print("Training required")
else:
    print("Account inactive")
```




**The inner decision only matters after the outer decision succeeds.**

CHOICE

# Nested or flat? Both can be correct

```
if account_active:
    if training_complete:
        print("Access granted")
    else:
        print("Training required")
else:
    print("Account inactive")
```

```
if account_active and training_complete:
    print("Access granted")
elif account_active:
    print("Training required")
else:
    print("Account inactive")
```

nested: mirrors step-by-step thinking

flat: fewer levels of indentation

**Pick the version that makes the decision process easiest to explain.**

DISTINCTION

# Independent checks are not an elif chain

```
if error_count > 0:
    print("Errors detected")

if missing_fields > 0:
    print("Missing fields detected")

if duplicate_count > 0:
    print("Duplicates detected")
```

**Separate if statements**

- Each condition is checked.
- Multiple messages can print.
- Use when multiple problems may exist at the same time.

```
if severity == "high":
    action = "reject"
elif severity == "medium":
    action = "review"
else:
    action = "accept"
```

**Use if / elif when choosing one outcome from a set of alternatives.**

# Short-circuit evaluation, at a beginner level

### and can stop early

```
if account_active and training_complete:
    print("Access granted")
```

If account_active is False, Python already knows the whole and expression cannot be True.

**False and ... → False**

### or can stop early

```
if is_admin or has_required_role:
    print("Allowed")
```

If is_admin is True, Python already knows the whole or expression is True.

**True or ... → True**

**Students do not need to exploit this yet; just know conditions are checked left to right.**

# Use a decision table before the code

| Active | Approved | Errors | Result |
|---|---|---|---|
| Yes | Yes | 0 | Ready |
| Yes | No | 0 | Awaiting approval |
| Yes | Yes | More than 0 | Review |
| No | Any | Any | Inactive |

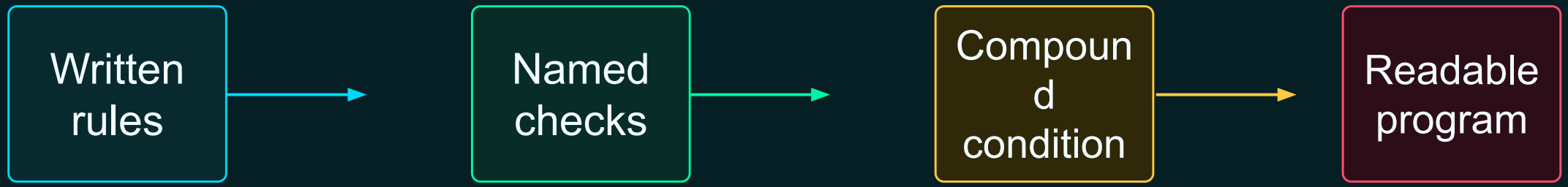
**Why tables help**

- Clarify the possible outcomes.
- Find missing cases before coding.
- Decide which checks come first.
- Explain the logic to another person.

**When logic gets messy, step away from Python and draw the rules.**

TRANSLATI ON

# Policy language → named Boolean variables → decision




```
has_supported_type = file_extension == "csv" or file_extension == "json"
has_valid_size = file_size > 0 and file_size <= maximum_size
can_process = has_supported_type and has_valid_size and is_approved
```

**Good Boolean names make conditions self-documenting.**

# File acceptance rules

**A file is accepted when...**

- The file is CSV or JSON.
- The file size is greater than zero.
- The file size is below the maximum.
- Approval has been received.
- It is not encrypted with an unsupported method.

```
has_supported_type = (
    file_extension == "csv"
    or file_extension == "json"
)

has_valid_size = file_size > 0 and file_size <=
maximum_size
has_supported_encryption = not
unsupported_encryption

can_accept_file = (
    has_supported_type
    and has_valid_size
    and is_approved
    and has_supported_encryption
)
```

**Write the checks first. Then combine them.**

# Equipment readiness rules

**Equipment is ready when...**

- Equipment is active.
- Inspection is complete.
- Operating hours are below the limit.
- No critical fault is present.
- Temperature is within the acceptable range.

```
is_active = status == "active"
has_inspection = inspection_complete
is_within_hours = operating_hours < maximum_hours
has_no_critical_fault = not critical_fault
is_temperature_ok = 60 <= temperature <= 80

is_ready = (
    is_active
    and has_inspection
    and is_within_hours
    and has_no_critical_fault
    and is_temperature_ok
)
```

```
if is_ready:
    print("Equipment ready")
else:
    print("Equipment not ready")
```

# User access rules

```
has_active_account = account_active
has_training = training_complete
has_role = user_role == "operator" or user_role ==
"analyst"
has_override = is_administrator
system_available = not maintenance_mode

can_access = (
    has_active_account
    and has_training
    and (has_role or has_override)
    and system_available
)
```

**Notice the grouped condition**

The user needs the required role OR administrator status.

That grouped result is one piece of the larger access decision.

**(has_role or has_override)**

**Parentheses make mixed and/or logic obvious.**

PRACTICE

# Print every intermediate condition

**Do not jump straight to the final answer**

**Students should print each named Boolean so they can explain why the final decision is True or False.**

```
print(f"Supported type: {has_supported_type}")
print(f"Valid size: {has_valid_size}")
print(f"Approved: {is_approved}")
print(f"Can process: {can_process}")
```

**Debugging benefit**

- The final result is not a mystery.
- Students can find which check failed.
- The program becomes easier to discuss with a teammate.

APPLIED CHALLENGE

# Data-publication workflow

### Workflow inputs

- Dataset status
- Data owner approval
- Security review completion
- Error count
- Missing-value percentage
- Restricted fields present?
- User has publishing role?

### Possible outcomes

- Publish
- Awaiting approval
- Security review required
- Data-quality review required
- Access denied
- Dataset inactive

**The hard part is not typing Python. The hard part is deciding the rules clearly.**

# Start with a decision table

| Condition | Question | Possible result |
|---|---|---|
| Publishing role? | Can this user publish at all? | Access denied |
| Dataset active? | Is the dataset still in use? | Dataset inactive |
| Owner approved? | Did the owner approve release? | Awaiting approval |
| Security reviewed? | Was security review completed? | Security review required |
| Quality OK? | Errors and missing values acceptable? | Data-quality review required |
| Everything OK? | All gates passed? | Publish |

**Evaluate blocking conditions first; publish only after the gates pass.**

APPLIED CHALLENGE

# Translate the table into code

```
has_publish_access = user_has_publish_role
is_dataset_active = dataset_status == "active"
has_owner_approval = owner_approved
has_security_review = security_review_complete
has_good_quality = error_count == 0 and missing_value_percentage <= 5
has_no_restricted_fields = not contains_restricted_fields

if not has_publish_access:
    result = "access denied"
elif not is_dataset_active:
    result = "dataset inactive"
elif not has_owner_approval:
    result = "awaiting approval"
elif not has_security_review or not has_no_restricted_fields:
    result = "security review required"
elif not has_good_quality:
    result = "data-quality review required"
else:
    result = "publish"
```

**Why this order?**

- Access is checked before wasting effort.
- Inactive datasets stop early.
- Approval and security are gates.
- Quality review comes before publishing.
- else is the success path after problems are ruled out.

# When conditions get too complex

**Simplify**
- Name important checks.
- Use parentheses for long expressions.
- Avoid clever one-liners.

**Trace**
- Print intermediate Booleans.
- Explain each path out loud.
- Test both True and False cases.

**Design**
- Use decision tables.
- Separate independent checks.
- Use nesting only when it clarifies dependency.

**Complex rules are manageable when you break them into named pieces.**

KNOWLEDGE CHECK

# Can you explain these decisions?

```
status = "active"
error_count = 0
is_approved = False

if status == "active" and error_count == 0 and
is_approved:
    print("Ready")
else:
    print("Not ready")
```

**Questions**

- What prints?
- Which condition prevents Ready?
- How would you make the output more helpful?

**Answer: Not ready. The approval check is False, so the combined and condition is False.**

**If a final Boolean surprises you, print the pieces.**

WRAP

# What students should leave with

## Compound logic lets Python make realistic decisions from multiple facts.

- and requires every condition to pass.
- or allows acceptable alternatives.
- not flips a condition.
- Nested if statements are best for dependent decisions.
- Decision tables help convert policy language into code.
- Readable Boolean names make programs easier to debug.

**Next: use decisions repeatedly with loops and collections.**